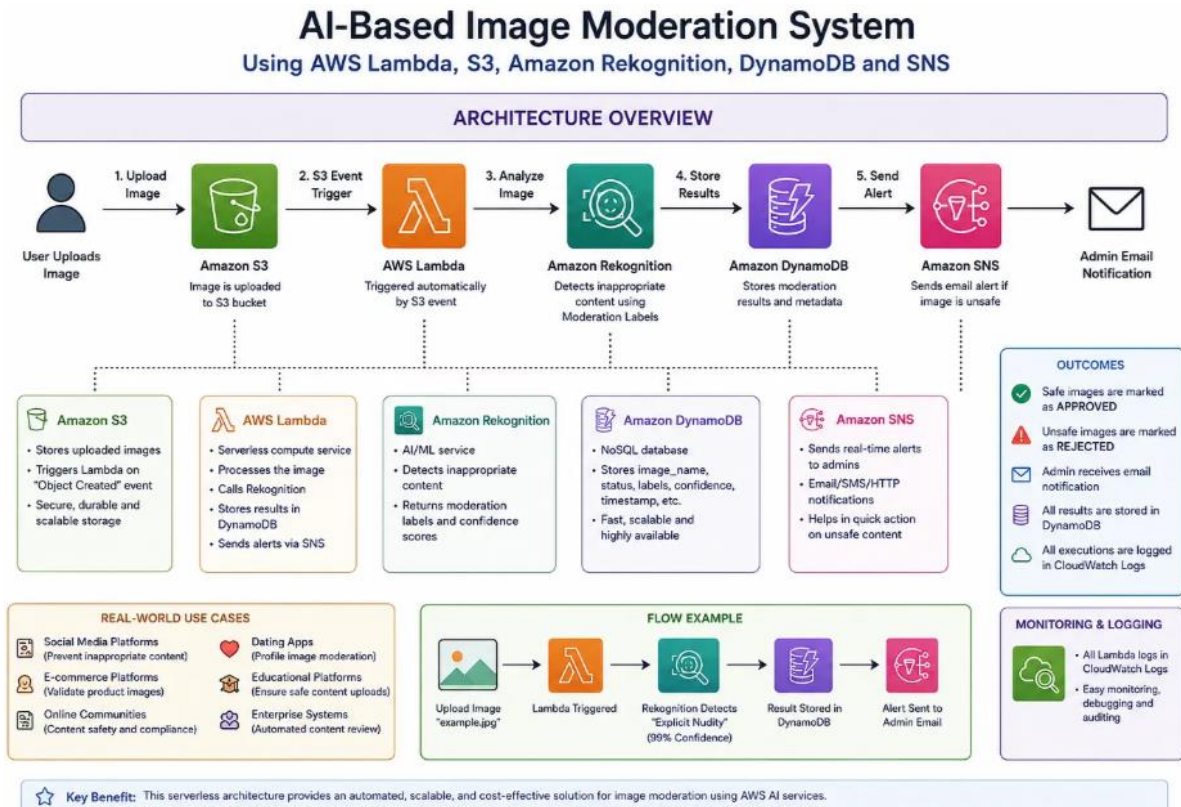


AI – Based Image Moderation System

- As organizations increasingly rely on user-generated content, manually reviewing uploaded images becomes time-consuming and difficult to scale.
- In this project, I built an **AI-powered Image Moderation System** using AWS serverless services.



What is Moderation Analysis?

Moderation Analysis is the process of automatically examining images or videos to identify content that may violate platform policies or community guidelines.

In your AWS project, Amazon Rekognition performs moderation analysis by checking whether an uploaded image contains:

- Explicit nudity
- Suggestive content
- Violence
- Graphic content
- Drugs
- Weapons

The solution automatically analyzes images uploaded to Amazon S3, detects potentially unsafe content using Amazon Rekognition, stores moderation results in DynamoDB, sends notifications through SNS, and records execution logs in CloudWatch.

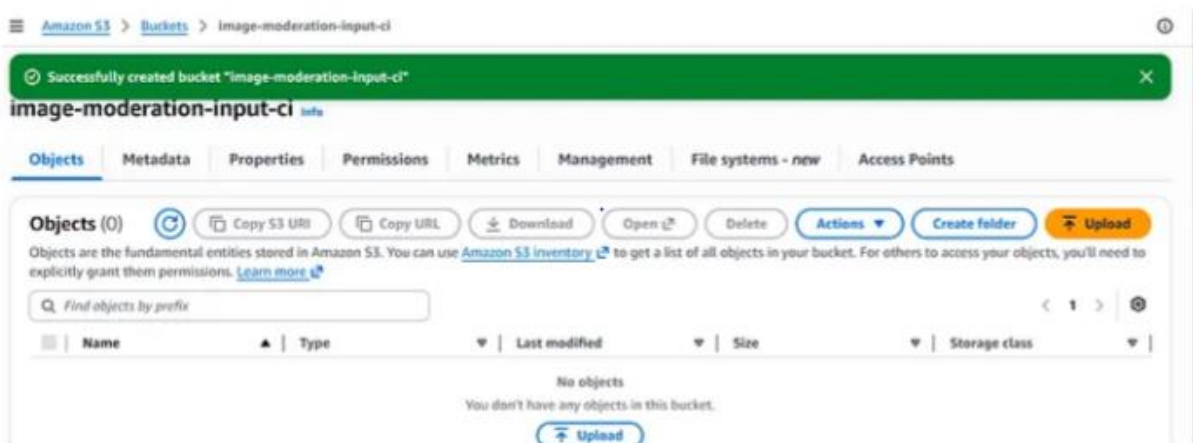
This project demonstrates how multiple AWS services can be integrated to build a scalable, event-driven architecture without managing any servers.

Services Used in This Project

- AWS Lambda - Serverless compute service that executes code in response to events.
- Amazon S3 - Object storage service used to store uploaded images.
- Amazon Rekognition - AI-powered image analysis service used to detect unsafe or inappropriate content.
- Amazon DynamoDB - Fully managed NoSQL database used to store image moderation results.
- Amazon SNS - Messaging and notification service used to send alerts when unsafe content is detected.
- Amazon CloudWatch - Monitoring and logging service used to track Lambda execution and troubleshoot issues.
- IAM - Identity and Access Management service used to securely grant permissions between AWS services.

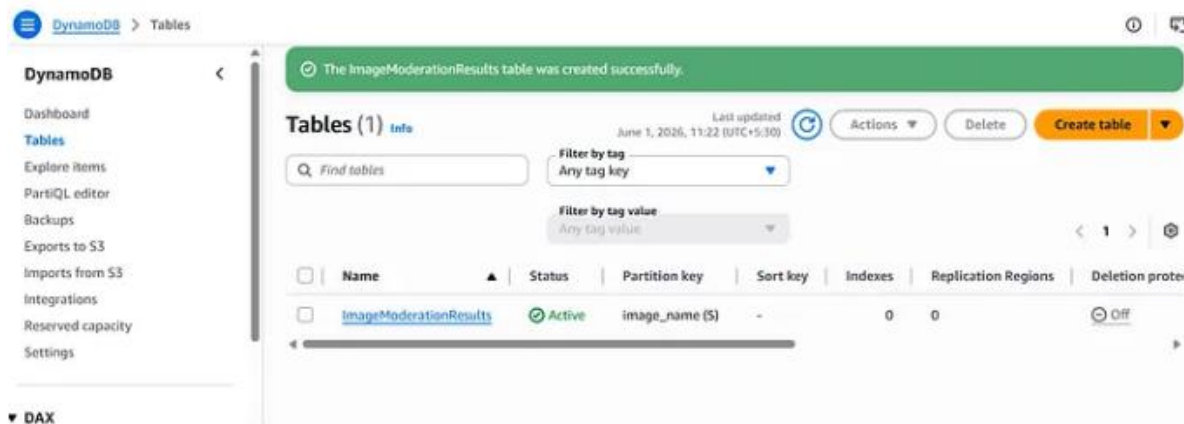
Started by creating an Amazon S3 bucket that will act as the image upload location. In this project, whenever a user uploads an image to this bucket, it will automatically trigger an AWS Lambda function for moderation analysis.

Step 1: Create S3 Bucket



Created an Amazon S3 bucket to store user-uploaded images. This bucket acts as the entry point of the image moderation workflow and triggers the Lambda function whenever a new image is uploaded.

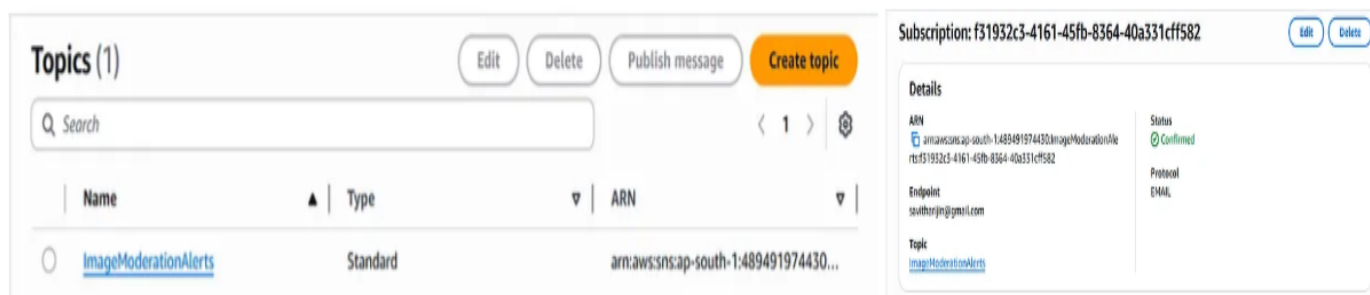
Step 2: Create DynamoDB Table



Created a DynamoDB table to store the moderation results. Each uploaded image will have an entry containing details such as image name, status, detected moderation labels, confidence score, and timestamp.

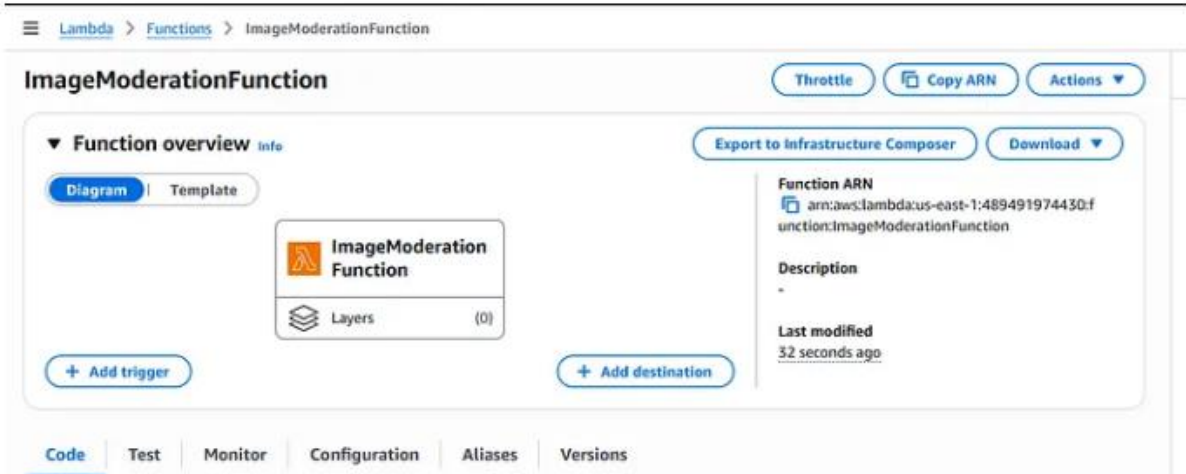
- The DynamoDB table acts as the central repository for storing image moderation results generated by the Lambda function.
- Each time an image is uploaded and analyzed by Amazon Rekognition, the moderation outcome is stored in DynamoDB for auditing, reporting, and future reference.
- Instead of re-analyzing images every time, the application can retrieve moderation results directly from DynamoDB, making the solution faster and more efficient.

Step 3: Create SNS Topic



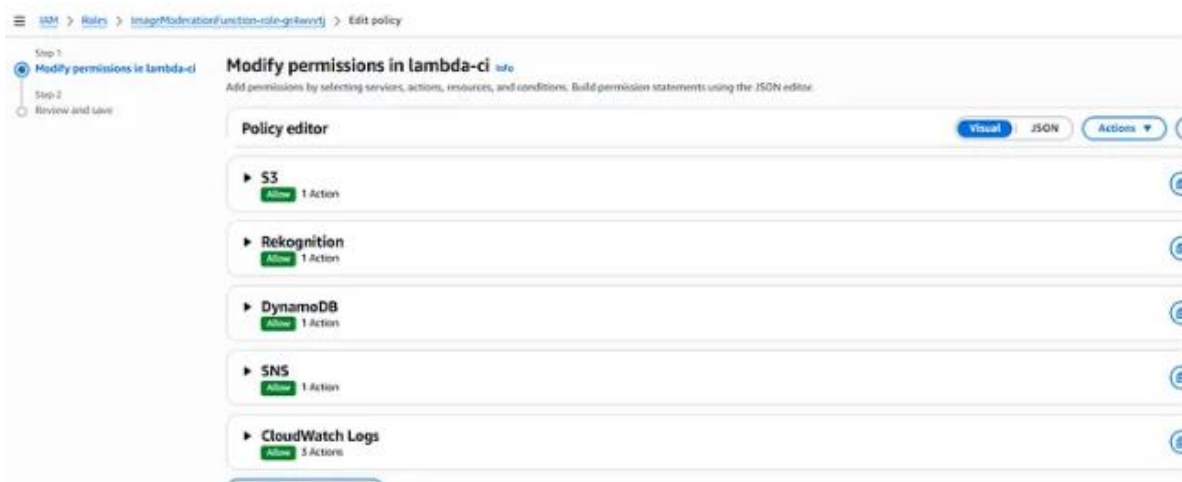
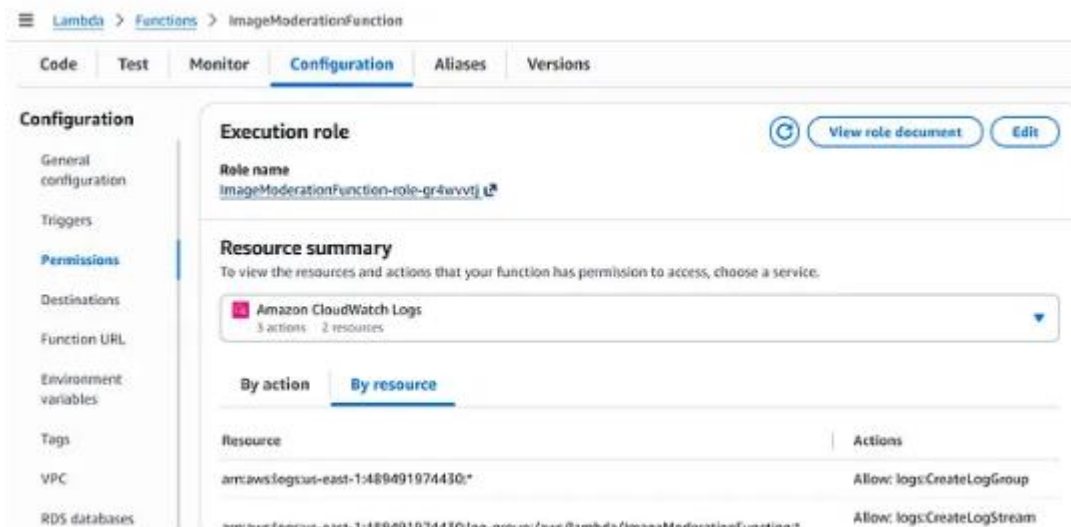
Create an SNS topic to send email notifications when an uploaded image is detected as unsafe. This helps admins receive alerts immediately and take quick action.

Step 4: Create Lambda Function



Created a Lambda function that acts as the processing engine of the image moderation workflow.

Step 5: Add IAM Permissions to Lambda Role

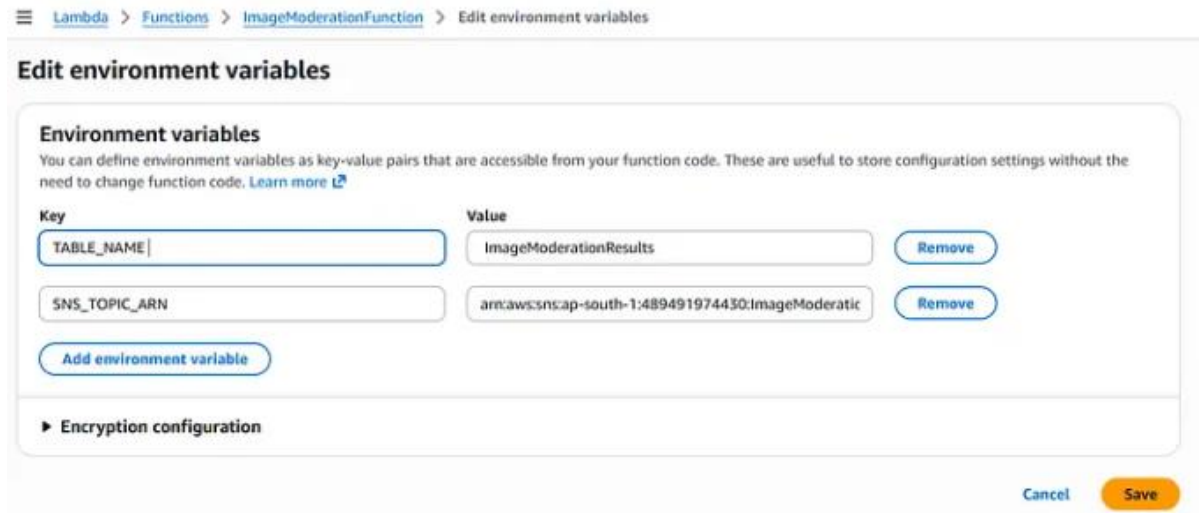


Updated the Lambda execution role with permissions to read images from S3, call Amazon Rekognition, store results in DynamoDB, publish SNS alerts, and write logs to CloudWatch.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3ReadAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::image-moderation-input-*/*"
    },
    {
      "Sid": "RekognitionAccess",
      "Effect": "Allow",
      "Action": [
        "rekognition:DetectModerationLabels"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DynamoDBAccess",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:*:*:table/ImageModerationResults"
    },
    {
      "Sid": "SNSPublishAccess",
      "Effect": "Allow",
      "Action": [
        "sns:Publish"
      ],
      "Resource": "*"
    },
    {
      "Sid": "CloudWatchLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

Step 6: Add Environment Variables

Configuration → Environment variables → Edit

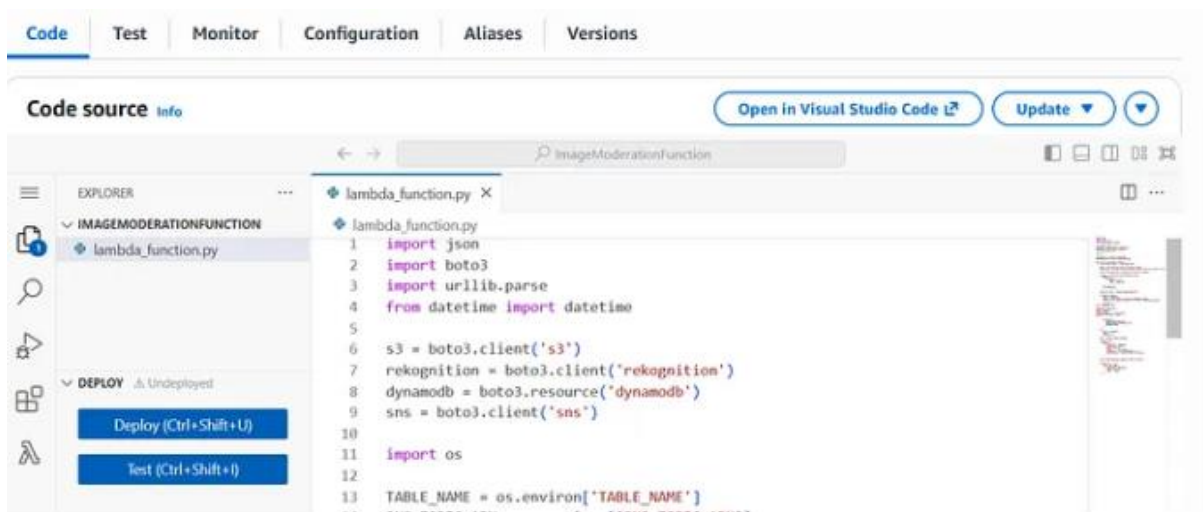


Configured Lambda environment variables to store the DynamoDB table name and SNS topic ARN. This avoids hardcoding important values directly inside the function code.

What are Environment Variables in Lambda?

Environment variables are key-value pairs stored outside your code that Lambda can access during execution.

Step 7: Add Lambda Code



Added Python code to the Lambda function. The function receives the S3 upload event, sends the image to Amazon Rekognition, stores the result in DynamoDB, and sends an SNS alert if unsafe content is detected.

```
import json
import boto3
import urllib.parse
from datetime import datetime

s3 = boto3.client('s3')
rekognition = boto3.client('rekognition')
dynamodb = boto3.resource('dynamodb')
sns = boto3.client('sns')

TABLE_NAME = os.environ['TABLE_NAME']
```

```

import os

TABLE_NAME = os.environ['TABLE_NAME']
SNS_TOPIC_ARN = os.environ['SNS_TOPIC_ARN']

def lambda_handler(event, context):
    print("Received event: ", json.dumps(event))

    bucket = event['Records'][0]['s3']['bucket']['name']
    image_name = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

    print(f"Processing image: {image_name} from bucket: {bucket}")

    response = rekognition.detect_moderation_labels(
        Image={
            'S3Object': {
                'Bucket': bucket,
                'Name': image_name
            }
        },
        MinConfidence=70
    )

    moderation_labels = response['ModerationLabels']

    if moderation_labels:
        status = "REJECTED"
        labels = [label['Name'] for label in moderation_labels]
        confidence = max([label['Confidence'] for label in moderation_labels])

        message = f"""
        Unsafe image detected!

        Image Name: {image_name}
        Bucket: {bucket}
        Status: {status}
        Detected Labels: {', '.join(labels)}
        Confidence: {confidence:.2f}%
        """

        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject="Image Moderation Alert",
            Message=message
        )

    else:
        status = "APPROVED"
        labels = []
        confidence = 0

    table = dynamodb.Table(TABLE_NAME)

    table.put_item(
        Item={
            'image_name': image_name,
            'bucket_name': bucket,
            'status': status,
            'labels': ', '.join(labels),
            'confidence': str(round(confidence, 2)),
            'processed_time': datetime.utcnow().isoformat()
        }
    )

    print(f"Image moderation completed. Status: {status}")

```



```

return {
  'statusCode': 200,
  'body': json.dumps({
    'image_name': image_name,
    'status': status,
    'labels': labels
  })
}

```

Step 8: Add S3 Trigger to Lambda

Choose:

Source: S3

Bucket: image-moderation-input-yourname

Event type: All object create events

ImageModerationFunction Throttle Copy ARN Actions

Function overview Info

Diagram Template

ImageModeration Function

Layers (0)

+ Add trigger + Add destination

Function ARN
arn:aws:lambda:us-east-1:489491974430:function:ImageModerationFunction

Description
-

Last modified
2 minutes ago

Add trigger

Trigger configuration Info

S3 aws asynchronous storage

Bucket
Choose or enter the ARN of an S3 bucket that serves as the event source. The bucket must be in the same region as the function.
s3/image-moderation-input-ci × 🔄
Bucket region: ap-south-1

Event types
Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.
All object create events ×

Prefix - optional
Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any special characters must be URL encoded.
e.g. images/

Suffix - optional
Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any special characters must be URL encoded.
e.g. .jpg

Recursive invocation
If your function writes objects to an S3 bucket, ensure that you are using different S3 buckets for input and output. Writing to the same bucket increases the risk of creating a recursive invocation, which can result in increased Lambda usage and increased costs. [Learn more](#)

☐ I acknowledge that using the same S3 bucket for both input and output is not recommended and that this configuration can cause recursive invocations, increased Lambda usage, and increased costs.

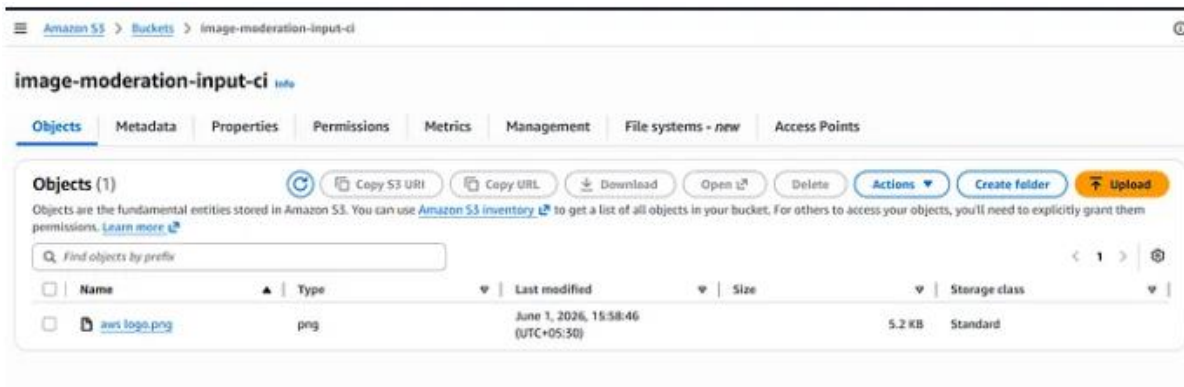
Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Cancel Add

Configured S3 as the event source for Lambda. Now, every new image upload to the S3 bucket automatically invokes the Lambda function.

Step 9: Upload Test Image

Upload a normal image first.



Uploaded a sample image to the S3 bucket. This upload event automatically triggered the Lambda function in the background.

Step 10: Check CloudWatch Logs

Lambda → Monitor → View CloudWatch logs

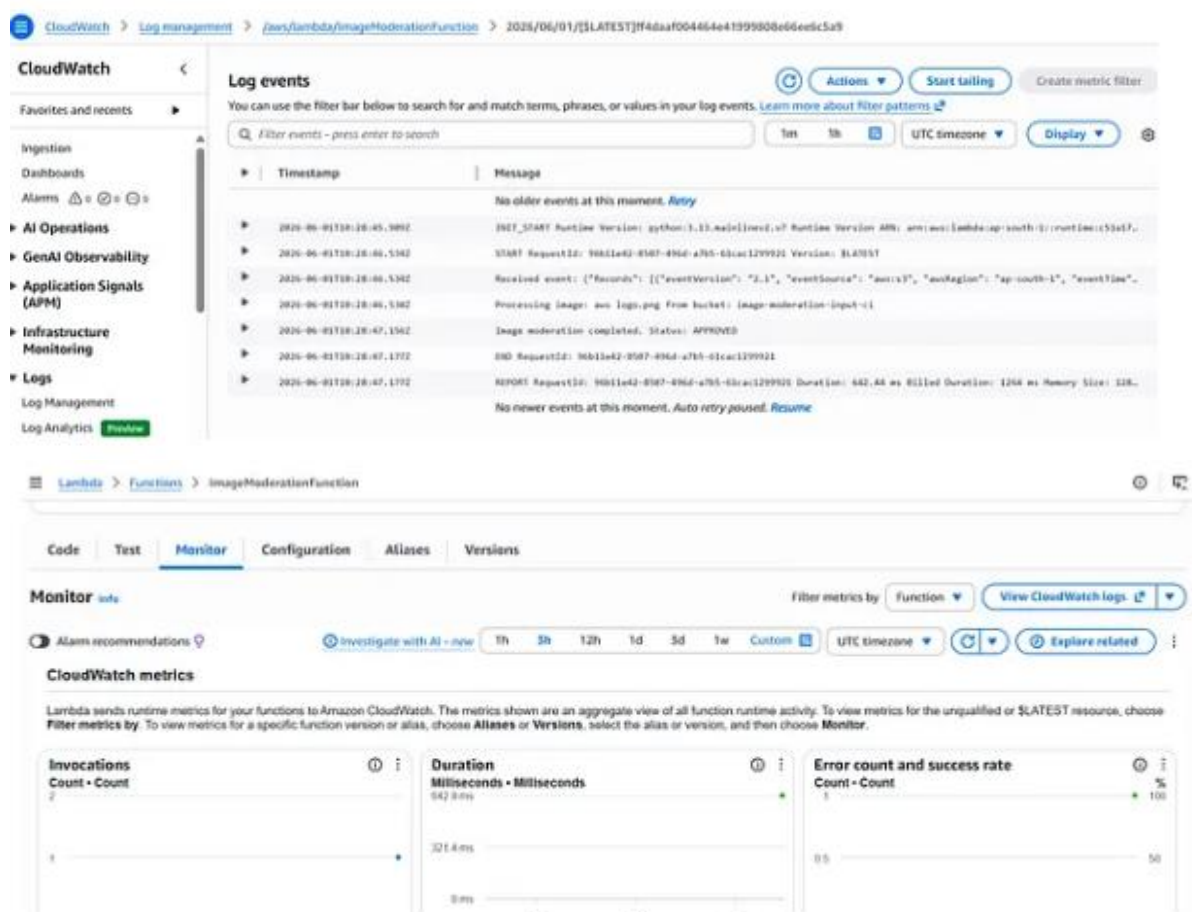
Open latest log stream.

You should see:

Processing image

Image moderation completed

Status: APPROVED



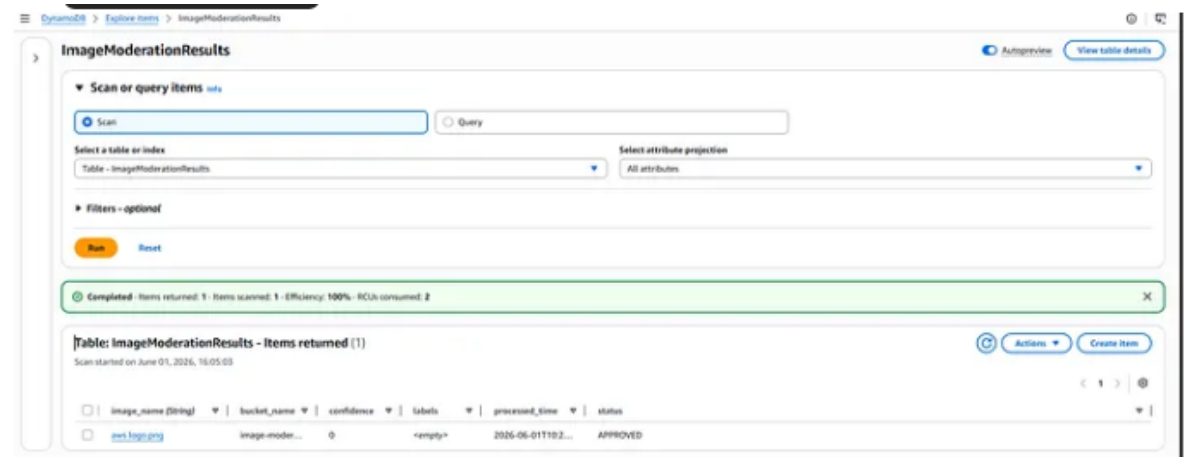
Verified the Lambda execution logs in CloudWatch. The logs confirm that the image was processed successfully and a moderation status was generated.

Step 11: Check DynamoDB Result

DynamoDB → Tables → ImageModerationResults → Explore table items

You should see an item like:

image__name: nature.jpg
status: APPROVED
labels:
confidence: 0



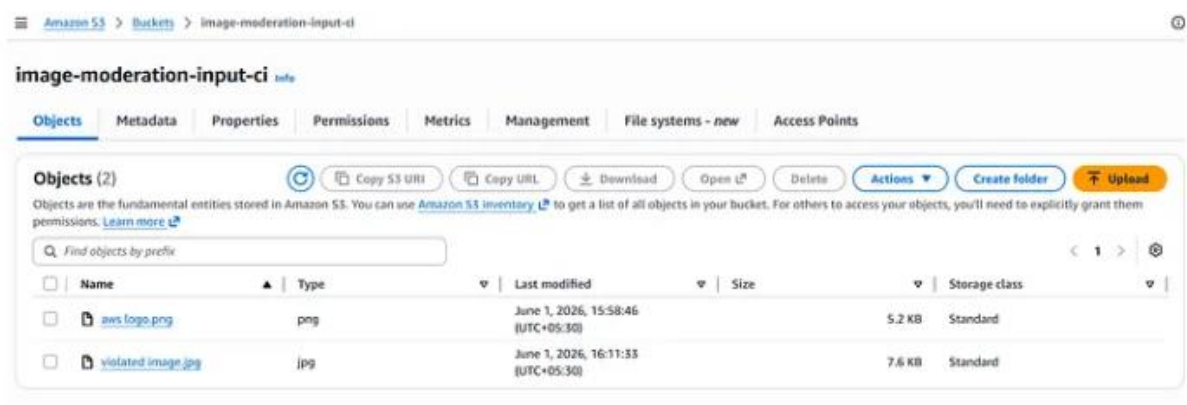
The image moderation result was stored successfully in DynamoDB. This table acts as an audit record for all uploaded images.

Attribute	Value	Meaning
image_name	aws logo.png	The image that was uploaded to S3 and processed by Lambda
bucket_name	image-moder...	The S3 bucket where the image was stored
confidence	0	Rekognition did not find any unsafe content, so no moderation confidence score was returned
labels	<empty>	No moderation labels were detected (such as Explicit Nudity, Violence, Drugs, etc.)
processed_time	2026-06-01T10:2...	Timestamp when Lambda completed the moderation analysis
status	APPROVED	Image was classified as safe

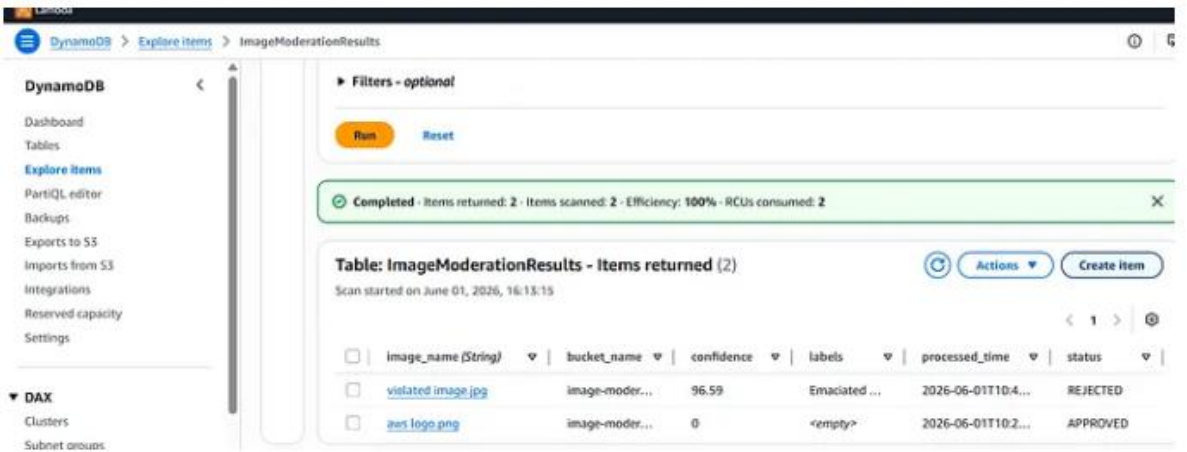
Step 12: Test Unsafe Image Scenario

You can test with an image that Rekognition may detect as unsafe.

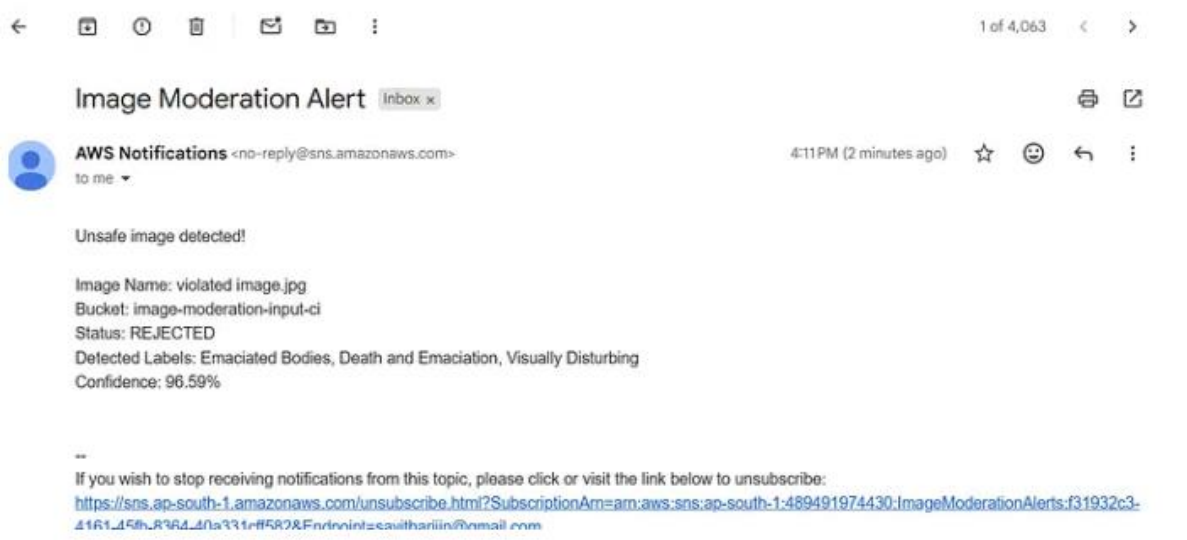
If Rekognition detects unsafe content, Lambda marks the image as REJECTED and sends an SNS email alert.



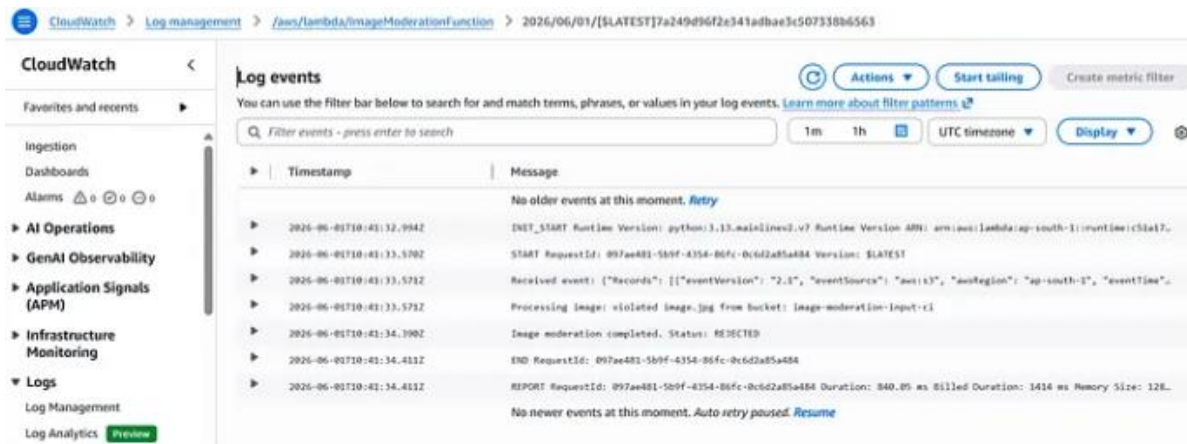
Added an inappropriate image.



DynamoDB table showing the reject.



Received an SNS notification alerting unsafe image upload.



Log Stream records the reject.

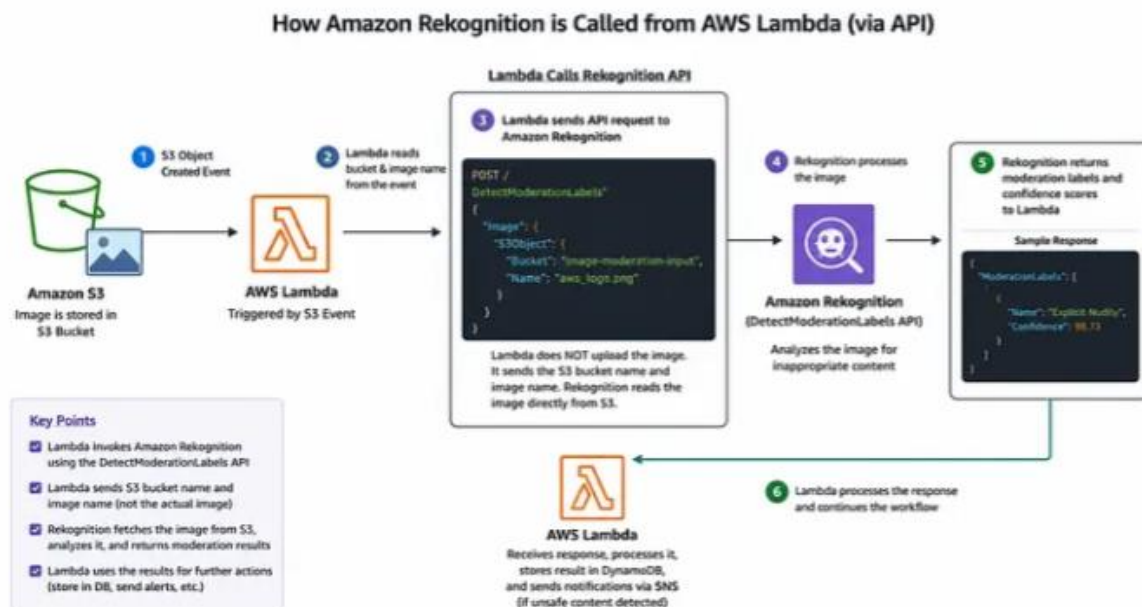
Will Rekognition console show my processed image history?

Usually, **no**.

Amazon Rekognition is an API-based service.

Amazon Rekognition was integrated into this project through the Lambda function. When an image was uploaded to S3, Lambda was triggered automatically and passed the image details to Amazon Rekognition using the `detect_moderation_labels()` API. Rekognition analyzed the image and returned moderation labels, confidence scores, and safety status back to Lambda. Lambda then stored this result in DynamoDB and triggered an SNS alert if unsafe content was detected.

Press enter or click to view image in full size



What starts as a simple image upload ultimately triggers a complete automated workflow involving storage, serverless compute, artificial intelligence, database persistence, notifications, and monitoring - showcasing the true potential of cloud-native application design on AWS.

